

Highlights

- Adaptive multi-kernel clustering is integrated into a clustering-based outlier detection framework to mitigate the influence of outliers on clustering structures.
 - Relative sample deviation and cluster rarity factor are defined to characterize different outlier properties.
 - A unified outlier scoring mechanism is constructed to jointly detect point outliers and clustered outliers.
-

Multi-kernel clustering-based outlier detection

Jiawei Cui^a, Zhong Yuan^{b,*}, Shihao Wang^a, Yinan Chai^c, Ze Zhang^a

^a College of Computer Science, Sichuan University, Chengdu, 610065, China

^b School of Artificial Intelligence, Sichuan University, Chengdu, 610065, China

^c College of Electrical Engineering, Sichuan University, Chengdu, 610207, China

ARTICLE INFO

Keywords:

Outlier detection
Multi-kernel learning
Clustering
Anomaly detection
K-means

ABSTRACT

Outlier Detection (OD) has attracted wide attention due to its important applications in financial risk control, cybersecurity, and medical diagnosis. Clustering-based outlier detection identifies outliers by analyzing the relationship between data and clustering structures, offering strong intuitive and interpretable results. However, the performance of such methods heavily depends on clustering results, and they often fail to adequately account for the interference of outliers during clustering, thereby degrading detection performance. Moreover, these methods typically struggle to simultaneously detect different types of outliers, leading to missed outliers. Multi-kernel learning adaptively integrates multiple kernel functions to map data into an appropriate feature space, thereby alleviating the influence of outliers on cluster structures, improving clustering quality, and enhancing outlier detection performance. Based on these observations, this paper proposes a multi-kernel clustering-based outlier detection method. Specifically, a kernel representation composed of multiple kernel functions is first constructed, and the weights of each kernel are adaptively learned during the clustering process to obtain the clustering partition of samples. Then, based on the clustering results, the clusters are divided into large and small clusters. By jointly considering cluster size and sample deviation, an outlier score is defined to enable the simultaneous detection of different types of outlier samples. Experimental comparisons with state-of-the-art methods show that the proposed method achieves competitive AUC performance. The code is publicly available online at <https://github.com/Tian468/MKCOD>.

1. Introduction

Kernel learning [1] is a powerful paradigm that extends the capability of traditional linear models by mapping data into a high-dimensional Hilbert space. This mechanism enables the effective handling of linearly non-separable data, thereby significantly improving model generalization and applicability. However, the performance of kernel-based models largely depends on the choice of kernel functions [2], and there is still a lack of systematic and well-established theoretical guidance for selecting optimal kernels. To address this issue, Multi-Kernel Learning (MKL) [3] introduces a predefined set of kernel functions and learns their optimal combination in a task-adaptive manner. In recent years, MKL has been widely applied in various domains, including image processing [4,5], object detection [6], and gene prediction [7]. In contrast, its application to outlier detection remains relatively underexplored [8,9].

Clustering is a fundamental concept in data mining and unsupervised learning [10,11], and has been widely studied and applied in fields such as pattern recognition and image analysis [12,13]. In recent years, a large number of clustering-based outlier detection methods have been proposed [14,15]. These methods identify outliers by analyzing the relationship between data instances and cluster

* Corresponding author.

E-mail address: yuanzhong@scu.edu.cn (Z. Yuan).

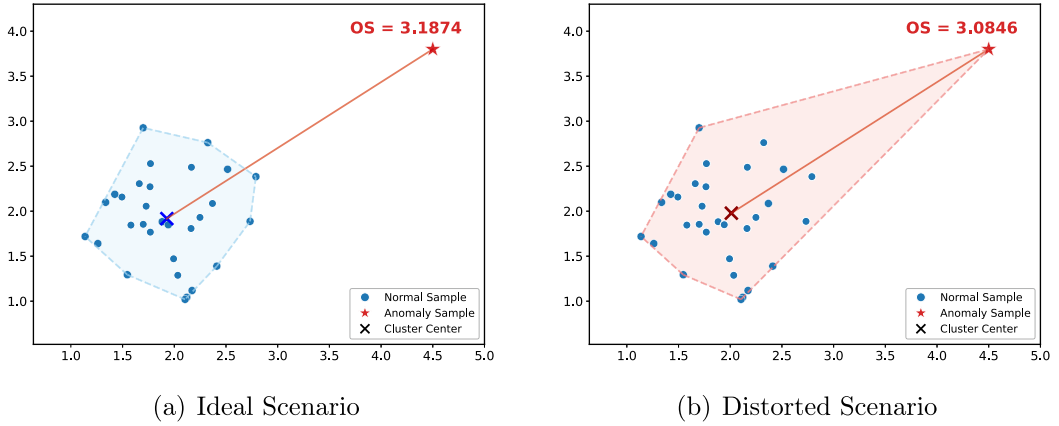


Fig. 1. Impact of outliers on cluster structure and outlier scores. OS denotes the outlying score, computed based on the distance between a sample and its corresponding cluster centroid.

structures, and typically regard samples that deviate from the overall clustering pattern as outliers. Based on different underlying assumptions, existing clustering-based outlier detection methods can be broadly categorized into three groups [16].

The first category of methods assumes that normal data instances belong to some cluster, whereas outliers do not belong to any cluster [17,18]. Based on this assumption, clustering algorithms that do not force all samples to be assigned to a cluster are typically applied, and data instances not assigned to any cluster are considered outliers. The second category assumes that normal data instances lie close to their nearest cluster centroid, while outliers are far from their nearest centroid [19]. These methods usually first perform clustering and then compute the distance between each instance and its nearest cluster centroid as the outlying score. The main limitation of these two categories is that they are primarily designed for point outliers [16]. When outliers aggregate in the feature space and form clustered outliers, these methods often fail to identify them effectively, leading to degraded detection performance. The third category of methods assumes that normal data instances belong to large and dense clusters, whereas outliers either form small clusters or reside in sparse clusters [20]. Based on this assumption, instances belonging to clusters with sizes or densities below a given threshold are identified as outliers. This category of methods alleviates, to some extent, the limitation of detecting clustered outliers.

However, from an overall perspective, the above three categories of methods generally treat outlier detection as a by-product of the clustering process, without explicitly considering the impact of outliers on the cluster structure itself. For example, in K-means-based methods, the presence of outliers may shift cluster centroids, thereby distorting the clustering structure and degrading detection performance. Fig. 1 illustrates this effect on both cluster structure and outlying scores. In this illustrative example, OS denotes the outlying score. It is computed as the Euclidean distance between the outlier sample and its assigned cluster centroid. In the ideal scenario (Fig. 1(a)), cluster centroids are not influenced by outliers, and outliers, being far from the centroids, receive high outlying scores ($OS = 3.1874$) and are correctly detected. In contrast, in practical scenarios (Fig. 1(b)), outliers participate in centroid computation, causing the centroids to shift toward the outliers. This shift reduces the outlying scores ($OS = 3.0846$), weakens the separability between outliers and normal instances, and ultimately leads to degraded detection performance.

To address the above issues, this paper proposes a multi-kernel clustering-based outlier detection algorithm. The method aims to suppress the influence of outliers on the clustering process and to enable unified detection of different types of outliers. First, multiple kernel matrices are constructed and then centered and normalized to ensure comparability across kernels. Second, an adaptive multi-kernel clustering model is adopted as the clustering component. In this process, kernel weights are adaptively updated to reduce the distortion of cluster structures caused by outliers. Next, cluster sizes are analyzed to divide clusters into large and small ones. Finally, an outlier scoring model is constructed by defining cluster dispersion, relative sample deviation, and cluster rarity factor, which jointly quantify the degree of outlierness. The main contributions of this work are summarized as follows.

- (1) To mitigate the influence of outliers on clustering structures, this paper integrates adaptive multi-kernel clustering into a clustering-based outlier detection framework.
- (2) To characterize different outlier properties, this paper defines relative sample deviation and cluster rarity factor.
- (3) To jointly detect point outliers and clustered outliers, this paper constructs a unified outlier score.

The remainder of this paper is organized as follows. Section 2 reviews related work. Section 3 introduces the basic concepts of the kernel k-means clustering algorithm. Section 4 presents the proposed multi-kernel clustering-based outlier detection method and describes the corresponding algorithm. Section 5 evaluates the effectiveness of the proposed method through extensive experimental comparisons. Finally, Section 6 concludes the paper and discusses directions for future work.

Table 1
Main notations of this paper.

Symbol	Description	Symbol	Description
$\mathbf{X}$	Data matrix	$\mathbf{K}^{(v)}$	The v -th base kernel matrix
$\mathbf{x}_i$	The i -th sample	θ	Kernel weight vector
c	Number of clusters	$\mathbf{K}_\theta$	Combined kernel matrix
$C(\mathbf{x})$	Cluster assignment of sample $\mathbf{x}$	$\mathbf{H}$	Cluster indicator matrix
α	Cumulative ratio threshold	μ_{C_j}	Center of cluster C_j
S	Set of small clusters	$d(\mathbf{x})$	Relative deviation of sample $\mathbf{x}$
L	Set of large clusters	$r(\mathbf{x})$	Cluster rarity factor of sample $\mathbf{x}$
σ_{C_j}	Dispersion of cluster C_j	$s(\mathbf{x})$	Outlier score of sample $\mathbf{x}$

2. Related work

This section reviews research work related to this study, focusing on multi-kernel clustering methods and clustering-based outlier detection methods. The two categories are introduced separately below.

2.1. Multi-kernel clustering methods

Kernel k-means maps data into a high-dimensional feature space via kernel functions and performs k-means clustering in that space to capture nonlinear cluster structures. Schölkopf et al. [21] and Zha et al. [22] showed that kernelized k-means can produce more reasonable clustering results than Euclidean k-means on complex data distributions. Du et al. [23] pointed out that the clustering performance of kernel k-means is highly sensitive to kernel selection, and that the optimal kernel is difficult to determine in unsupervised settings, which affects clustering stability.

To alleviate the kernel selection problem and improve clustering quality, multi-kernel clustering methods have been proposed [24]. Yu et al. [25], Huang et al. [26], and Gönen and Alpaydın [24] constructed weighted combinations of multiple predefined kernels and jointly learned kernel weights and cluster assignments during the clustering process. Bang et al. [27] noted that most existing multi-kernel k-means methods adopt alternating optimization strategies to learn kernel weights and clustering structures. Experimental results reported by Yu et al. [25] and Gönen and Alpaydın [24] show that multi-kernel clustering achieves better clustering performance than single-kernel methods on various datasets. Based on these studies, further research has investigated the stability of multi-kernel clustering under outlier and noise interference. Du et al. [23] addressed the sensitivity of squared loss to anomalous samples and proposed a robust multi-kernel k-means method based on the $\ell_{2,1}$ norm. From the perspective of kernel weight learning, Bang et al. [27] proposed a multi-kernel k-means method based on min-max optimization. These two lines of work improve the stability of multi-kernel clustering in the presence of anomalous samples and noisy views from the perspectives of loss design and weight learning, respectively.

Multi-kernel clustering methods improve quality and stability by integrating multiple kernels. However, most studies focus only on unsupervised clustering tasks. Research on applying these methods to outlier detection is limited, especially work on unifying multi-kernel clustering and outlier detection.

2.2. Clustering-based outlier detection methods

Early clustering-based outlier detection methods define outliers as samples that cannot be assigned to any cluster. DBSCAN, proposed by Ester et al. [17], constructs clusters based on density reachability and labels samples in low-density regions as noise. OPTICS, introduced by Ankerst et al. [28], characterizes clustering structures under different density thresholds, thereby reducing the dependence on a single density parameter. ROCK, proposed by Guha et al. [29], builds clusters based on link relationships and treats samples that do not satisfy the minimum link requirement as outliers. Another line of work characterizes outliers according to their relationship with cluster centers. The k-means algorithm proposed by McQueen [30] has been widely adopted for outlier detection, where outlier scores are typically defined by the distance to the assigned cluster center or by the contribution to the clustering objective. To mitigate the influence of anomalous samples on centroid estimation, Chawla and Gionis [31] proposed KMeans-, which allows a subset of samples to be explicitly excluded during optimization. He et al. [20] further introduced CBLOF, which incorporates cluster size information and assigns higher outlier scores to samples belonging to small clusters based on both cluster size and center distance.

Clustering-based outlier detection methods identify outliers by modeling the relationship between samples and cluster structures. Most existing approaches are designed for a single type of outlier, making it difficult to detect different outlier types simultaneously. In addition, outlying samples participate in cluster structure learning and may interfere with centroid estimation and cluster formation, thereby degrading the reliability of outlier detection. Therefore, enabling unified detection of multiple outlier types while suppressing the influence of outlying samples on the clustering process remains a key research problem.

3. Preliminaries

This section introduces the basic formulation of kernel k-means clustering, which serves as the foundation for subsequent multi-kernel methods. The main notations used throughout this paper are summarized in Table 1.

Let $\mathbf{X} = [\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n] \in \mathbb{R}^{p \times n}$ denote the data matrix with n samples, where each column $\mathbf{x}_i \in \mathbb{R}^p$ denotes a sample. Let $\phi : \mathbb{R}^p \rightarrow \mathcal{F}$ be a nonlinear mapping induced by a kernel function, which maps samples into a reproducing kernel Hilbert space $\mathcal{F}$. Kernel k-means performs k-means clustering in the feature space.

Let $\mathbf{Z} = [z_{ij}] \in \{0, 1\}^{n \times c}$ be the cluster assignment matrix, where $z_{ij} = 1$ indicates that sample $\mathbf{x}_i$ is assigned to the j -th cluster and $\sum_{j=1}^c z_{ij} = 1$. The objective of kernel k-means is to minimize the within-cluster sum of squared distances in the feature space,

$$\sum_{j=1}^c \sum_{i=1}^n z_{ij} \left\| \phi(\mathbf{x}_i) - \boldsymbol{\mu}_j \right\|_2^2, \quad (1)$$

where the cluster center is $\boldsymbol{\mu}_j = \frac{1}{n_j} \sum_{i=1}^n z_{ij} \phi(\mathbf{x}_i)$ and $n_j = \sum_{i=1}^n z_{ij}$ denotes the cluster size.

Let $\mathbf{K} \in \mathbb{R}^{n \times n}$ be the kernel matrix with entries $K_{ij} = \langle \phi(\mathbf{x}_i), \phi(\mathbf{x}_j) \rangle$. By introducing the normalized cluster assignment matrix $\mathbf{H} = \mathbf{Z} \mathbf{D}^{-1/2}$ with $\mathbf{D} = \text{diag}(n_1, \dots, n_c)$, Zha et al. [22] showed that the above objective can be equivalently reformulated as the following trace minimization problem:

$$\min_{\mathbf{H} \in \mathbb{R}^{n \times c}} \text{tr}(\mathbf{K} - \mathbf{H}^T \mathbf{K} \mathbf{H}), \quad \text{s.t. } \mathbf{H}^T \mathbf{H} = \mathbf{I}_c. \quad (2)$$

The discrete constraint on $\mathbf{Z}$ is relaxed by solving the above problem in the continuous domain. The optimal solution is obtained from the eigenvectors corresponding to the c largest eigenvalues of $\mathbf{K}$ [32]. After obtaining the continuous solution $\mathbf{H}$, a hard clustering assignment can be recovered by applying k-means clustering to the rows of $\mathbf{H}$ [33].

4. Proposed outlier detection method

In this section, we present the proposed Multi-Kernel Clustering-based Outlier Detection (MKCOD) method, including multi-kernel clustering, outlier scoring, and the complete algorithm.

4.1. Multi-kernel clustering

As discussed in Section 1, existing clustering-based outlier detection methods usually ignore the influence of outlier samples on the clustering process, which leads to degraded detection performance. To alleviate this issue, we adopt a robust multi-kernel clustering algorithm. By adaptively learning kernel weights during clustering, the algorithm avoids excessive reliance on kernels that are sensitive to outliers, thereby reducing such effects.

We adopt the multi-kernel k-means clustering model proposed by Bang et al. [27] as the clustering component of our framework. This model jointly learns the clustering structure and kernel weights under a multi-kernel combination framework via a min-max optimization scheme. Before the iterative optimization, kernel matrices are preprocessed following the settings in [27]. Specifically, according to [21], each base kernel matrix is first centered using the kernel trick, $\mathbf{K}^{(v)} \leftarrow \mathbf{K}^{(v)} - \mathbf{J}_n \mathbf{K}^{(v)} - \mathbf{K}^{(v)} \mathbf{J}_n + \mathbf{J}_n \mathbf{K}^{(v)} \mathbf{J}_n$, where $\mathbf{J}_n = \mathbf{1}\mathbf{1}^T/n$. Then, each centered base kernel matrix is scaled to eliminate magnitude differences across kernels [34], $\mathbf{K}^{(v)} \leftarrow \mathbf{K}^{(v)} / \text{tr}(\mathbf{K}^{(v)})$. Subsequently, the combined kernel matrix is constructed via linear weighting as $\mathbf{K}_\theta = \sum_{v=1}^M \theta_v \mathbf{K}^{(v)}$, where $\theta = [\theta_1, \dots, \theta_M]^T$ denotes the kernel weight vector.

After preprocessing, the multi-kernel k-means clustering problem is formulated as the following min-max optimization problem:

$$\min_{\mathbf{H}} \max_{\boldsymbol{\theta}} \text{tr}(\mathbf{K}_\theta - \mathbf{H}^T \mathbf{K}_\theta \mathbf{H}) \quad \text{s.t. } \mathbf{H}^T \mathbf{H} = \mathbf{I}_c, \quad \theta \geq 0, \quad \boldsymbol{\theta}^T \boldsymbol{\theta} \leq 1. \quad (3)$$

Here, $\mathbf{H} \in \mathbb{R}^{n \times c}$ denotes the relaxed cluster indicator matrix.

The above problem is solved via an alternating optimization strategy. When the kernel weights $\boldsymbol{\theta}$ are fixed, the combined kernel matrix $\mathbf{K}_\theta$ is fixed, and the problem reduces to a standard kernel k-means clustering problem:

$$\min_{\mathbf{H}} \text{tr}(\mathbf{K}_\theta - \mathbf{H}^T \mathbf{K}_\theta \mathbf{H}) \quad \text{s.t. } \mathbf{H}^T \mathbf{H} = \mathbf{I}_c. \quad (4)$$

This problem can be solved by eigen-decomposition of $\mathbf{K}_\theta$, where the optimal solution is given by the eigenvectors corresponding to the top c largest eigenvalues.

When the cluster indicator matrix $\mathbf{H}$ is fixed, the optimization problem reduces to a subproblem with respect to the kernel weights $\boldsymbol{\theta}$:

$$\max_{\boldsymbol{\theta}} \sum_{v=1}^M \theta_v \text{tr}(\mathbf{K}^{(v)} - \mathbf{H}^T \mathbf{K}^{(v)} \mathbf{H}) \quad \text{s.t. } \boldsymbol{\theta} \geq 0, \quad \boldsymbol{\theta}^T \boldsymbol{\theta} \leq 1. \quad (5)$$

According to the derivation in [27], this subproblem admits a closed-form solution:

$$\theta_v = \frac{\text{tr}(\mathbf{K}^{(v)} - \mathbf{H}^T \mathbf{K}^{(v)} \mathbf{H})}{\sqrt{\sum_{j=1}^M (\text{tr}(\mathbf{K}^{(j)} - \mathbf{H}^T \mathbf{K}^{(j)} \mathbf{H}))^2}}, \quad v = 1, \dots, M. \quad (6)$$

The above two steps are alternated until convergence. Bang *et al.* proved that this alternating optimization procedure converges to a saddle point solution, and each subproblem admits an analytical solution. After convergence, a continuous cluster indicator matrix $\mathbf{H}$ is obtained, and discrete cluster assignments are recovered by applying k-means clustering to the row vectors of $\mathbf{H}$. By adaptively adjusting kernel weights, this multi-kernel clustering algorithm reduces the dependence on any single kernel and alleviates the influence of anomaly-sensitive kernels on the clustering results, thereby weakening the interference of outlier samples during clustering and providing a reliable clustering structure for subsequent outlier detection.

4.2. Outlier score

As discussed in Section 1, existing clustering-based outlier detection methods usually focus on a specific type of outliers, which makes it difficult to detect different types of outliers simultaneously. To address this issue, this section introduces a partition of clusters into large and small ones, and constructs an outlier scoring mechanism by characterizing cluster dispersion, cluster rarity factor, and relative sample deviation.

We first introduce the clustering results. Using the method described in Section 4.1, the samples in the data matrix $\mathbf{X}$ are clustered into c clusters, denoted by $C = \{C_1, C_2, \dots, C_c\}$. This partition satisfies $C_i \cap C_j = \emptyset$ ($i \neq j$) and $\bigcup_{j=1}^c C_j$ contains all samples. For any sample $\mathbf{x} \in \mathbb{R}^p$, $C(\mathbf{x})$ denotes the cluster to which $\mathbf{x}$ belongs, and $|C_j|$ denotes the size of cluster C_j .

Definition 1. Given a cluster set $C = \{C_1, C_2, \dots, C_c\}$ obtained from the clustering result, all clusters are sorted in descending order according to their sizes, such that $|C_1| \geq |C_2| \geq \dots \geq |C_c|$. Given a cumulative ratio threshold $\alpha \in (0, 1]$, let m be the smallest positive integer satisfying

$$|C_1| + |C_2| + \dots + |C_m| \geq \alpha n. \quad (7)$$

Then, the large-cluster set and the small-cluster set are defined as

$$L = \{C_i \mid 1 \leq i \leq m\}, \quad S = \{C_i \mid i > m\}, \quad (8)$$

respectively.

Definition 2. Given a cluster set $C = \{C_1, C_2, \dots, C_c\}$, let μ_{C_j} denote the center of cluster C_j . The dispersion of cluster C_j is defined as the average distance from samples in the cluster to its center, given by

$$\sigma_{C_j} = \frac{1}{|C_j|} \sum_{\mathbf{x} \in C_j} \|\mathbf{x} - \mu_{C_j}\|_2. \quad (9)$$

Here, σ_{C_j} characterizes the dispersion of samples within cluster C_j . When σ_{C_j} is large, samples in C_j are more dispersed, and outliers in the cluster tend to have larger distances to the cluster center; when σ_{C_j} is small, samples are more compact and the corresponding distances are relatively smaller. This quantity provides the basis for defining the relative sample deviation, thereby enabling the comparison of sample deviations across different clusters.

Definition 3. Given a sample $\mathbf{x} \in \mathbb{R}^p$, let $C(\mathbf{x})$ denote the cluster to which $\mathbf{x}$ belongs. The relative deviation of $\mathbf{x}$ is defined as

$$d(\mathbf{x}) = \begin{cases} \frac{\|\mathbf{x} - \mu_{C(\mathbf{x})}\|_2}{\sigma_{C(\mathbf{x})}}, & C(\mathbf{x}) \in L, \\ \frac{\|\mathbf{x} - \mu_{\ell(\mathbf{x})}\|_2}{\sigma_{\ell(\mathbf{x})}}, & C(\mathbf{x}) \in S, \end{cases} \quad (10)$$

where $\ell(\mathbf{x})$ denotes the index of the large cluster that is closest to the small cluster $C(\mathbf{x})$.

The relative deviation $d(\mathbf{x})$ measures the deviation of sample $\mathbf{x}$ with respect to a reference cluster center while accounting for the dispersion of the reference cluster. For samples in large clusters, the reference cluster is the cluster to which the sample belongs. For samples in small clusters, the reference cluster is chosen as the nearest large cluster.

The relative sample deviation is designed to measure the deviation of an individual sample from the normal data distribution, so as to identify point outliers. Large clusters contain more samples, and their centers can represent the main concentration of normal data. In contrast, small clusters contain only a limited number of samples, and their centers are usually not regarded as reliable representatives of normal data distribution. Therefore, for samples in small clusters, using the nearest large cluster as the reference provides a more reliable way to measure their deviation. In addition, determining the reference large cluster at the cluster level and reusing it for samples within the same cluster helps reduce repeated computations and simplify the overall computational complexity.

Definition 4. Given a sample $\mathbf{x} \in \mathbb{R}^p$, let $C(\mathbf{x})$ denote the cluster to which $\mathbf{x}$ belongs. The cluster rarity factor of sample $\mathbf{x}$ is defined as

$$r(\mathbf{x}) = \begin{cases} 1, & C(\mathbf{x}) \in L, \\ 1 + \log\left(\frac{|C_{\max}|}{|C(\mathbf{x})|}\right), & C(\mathbf{x}) \in S, \end{cases} \quad (11)$$

where $C_{\max}$ denotes the cluster with the largest size in C .

The cluster rarity factor is introduced to characterize the rarity of a cluster based on its size, so as to identify clustered outliers. As the cluster size decreases relative to the largest cluster, the corresponding rarity factor increases, indicating a higher likelihood that samples in the cluster are outliers. At this point, point outliers are characterized by the relative sample deviation, while clustered outliers are captured by the cluster rarity factor.

To further explain the cluster rarity factor defined in this paper, we use empirical probability estimation and self-information. Let n be the number of samples. The empirical probability of cluster C_j is $\hat{p}(C_j) = |C_j|/n$. A smaller cluster has a lower empirical probability and a higher rarity.

In information theory, the self-information of cluster C_j is $I(C_j) = -\log \hat{p}(C_j)$. Thus, the extra information of C_j relative to the largest cluster $C_{\max}$ is

$$I(C_j) - I(C_{\max}) = \log \frac{\hat{p}(C_{\max})}{\hat{p}(C_j)} = \log \frac{|C_{\max}|}{|C_j|}. \quad (12)$$

This equation shows that $\log(|C_{\max}|/|C_j|)$ characterizes the rarity of C_j relative to the largest cluster. Therefore, Definition 4 uses the logarithmic form to construct the cluster rarity factor. Compared with the linear ratio $|C_{\max}|/|C_j|$, the logarithmic form is still monotonic with respect to cluster size, but grows more slowly. Thus, it reduces the excessive rarity value assigned to extremely small clusters. In this paper, $\log(\cdot)$ denotes the natural logarithm.

In the following, a unified outlier score is defined to jointly identify these two types of outliers.

Definition 5. Given a sample $\mathbf{x} \in \mathbb{R}^p$, the outlier score of $\mathbf{x}$ is defined as

$$s(\mathbf{x}) = d(\mathbf{x}) \cdot r(\mathbf{x}). \quad (13)$$

Here, $d(\mathbf{x})$ and $r(\mathbf{x})$ denote the relative sample deviation and cluster rarity factor defined in Eqs. (10) and (11), respectively. A larger outlier score $s(\mathbf{x})$ indicates that the sample is more likely to be an outlier, either due to a larger deviation from the reference large-cluster center or due to belonging to a rarer cluster.

A special case occurs when $\alpha = 1$. In this case, $m = c$, so $L = C$ and $S = \emptyset$. Hence, $r(\mathbf{x}) = 1$ for every sample, and the outlier score reduces to $s(\mathbf{x}) = \|\mathbf{x} - \mu_{C(\mathbf{x})}\|_2 / \sigma_{C(\mathbf{x})}$, i.e., the distance from $\mathbf{x}$ to the center of its own cluster normalized by the dispersion of that cluster.

We provide a heuristic posterior-odds interpretation of the multiplicative form in Eq. (13). Let $Y \in \{0, 1\}$ be the class label, where $Y = 0$ denotes a normal sample and $Y = 1$ denotes an outlier. For a sample $\mathbf{x}$, let $D = d(\mathbf{x})$ and $R = r(\mathbf{x})$.

According to Bayes' rule, the posterior odds of $\mathbf{x}$ being an outlier are

$$\frac{P(Y = 1 | D, R)}{P(Y = 0 | D, R)} = \frac{P(D, R | Y = 1)}{P(D, R | Y = 0)} \cdot \frac{P(Y = 1)}{P(Y = 0)}. \quad (14)$$

Under a simplifying conditional-independence assumption, $P(D, R | Y) \approx P(D | Y)P(R | Y)$. Then,

$$\frac{P(Y = 1 | D, R)}{P(Y = 0 | D, R)} \propto \frac{P(D | Y = 1)}{P(D | Y = 0)} \cdot \frac{P(R | Y = 1)}{P(R | Y = 0)}. \quad (15)$$

This suggests that the product of the deviation term and the rarity term can be viewed as a heuristic approximation to the posterior odds. Since the prior odds $P(Y = 1)/P(Y = 0)$ are the same for all samples, they do not affect the score ranking.

In MKCOD, $d(\mathbf{x})$ measures the deviation from the reference cluster center, while $r(\mathbf{x})$ reflects cluster rarity. For samples in large clusters, $r(\mathbf{x}) = 1$, and the score depends on deviation. For samples in small clusters, $r(\mathbf{x}) > 1$, and the deviation is amplified by cluster rarity. Thus, the multiplicative score provides a practical way to combine point-level deviation and cluster-level rarity.

Definition 6. Let μ denote a predefined outlier threshold. For a sample $\mathbf{x} \in \mathbb{R}^p$, if

$$s(\mathbf{x}) > \mu, \quad (16)$$

then $\mathbf{x}$ is identified as an outlier.

The threshold μ is used to convert outlier scores into binary decisions. Following the contamination-based setting in outlier detection toolkits [35], when the expected outlier ratio ρ is available, the top $\lceil \rho n \rceil$ samples with the highest scores are identified as outliers, and μ is set as the corresponding score cutoff. In practical applications, ρ can be estimated from domain knowledge.

4.3. Corresponding algorithm

This section presents the complete MKCOD algorithm and analyzes its time complexity. MKCOD has two tunable hyperparameters: the number of clusters c and the cumulative ratio threshold α for separating large and small clusters.

In Algorithm 1, multiple base kernel matrices are first constructed and preprocessed, which requires $O(Mn^2d)$ time. Then, the cluster indicator matrix and kernel weights are updated alternately for I iterations. Specifically, updating the cluster indicator matrix with fixed kernel weights involves eigen-decomposition, with a time complexity of $O(In^3)$, while updating the kernel weights with fixed cluster indicators requires computing trace terms of base kernels, with a time complexity of $O(IMcn^2)$. Finally, cluster statistics are computed, and outlier scores are evaluated for all samples, which has a time complexity of $O(nc)$ and can be neglected. Overall, the total time complexity of MKCOD is $O(Mn^2d + In^3 + IMcn^2)$. The framework diagram of MKCOD is described in Fig. 2.

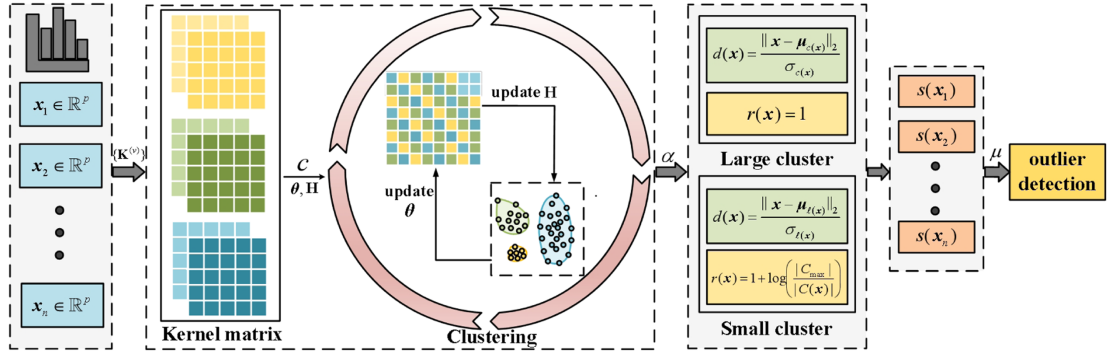


Fig. 2. The framework of MKCOD.

Algorithm 1: MKCOD.

Input: $\mathbf{X} = [\mathbf{x}_1, \dots, \mathbf{x}_n]$: data matrix, c : number of clusters, α : cumulative ratio threshold, $\{\mathbf{K}^{(v)}\}_{v=1}^M$: base kernel matrices
Output: s : outlier scores

- 1 $\mathbf{K}^{(v)} \leftarrow \mathbf{K}^{(v)} - \mathbf{J}_n \mathbf{K}^{(v)} - \mathbf{K}^{(v)} \mathbf{J}_n + \mathbf{J}_n \mathbf{K}^{(v)} \mathbf{J}_n$, $\mathbf{J}_n = \mathbf{1}\mathbf{1}^T/n$, $v = 1, \dots, M$;
- 2 $\mathbf{K}^{(v)} \leftarrow \mathbf{K}^{(v)} / \text{tr}(\mathbf{K}^{(v)})$, $v = 1, \dots, M$;
- 3 initialize θ , $\mathbf{K}_\theta$, $\mathbf{H}$;
- 4 **repeat**
- 5 Fix θ , update $\mathbf{H}$ by solving Eq. (4);
- 6 Fix $\mathbf{H}$, update θ by Eq. (6);
- 7 **until** convergence;
- 8 Apply k-means to the rows of $\mathbf{H}$ to obtain $\{C_1, \dots, C_c\}$;
- 9 Sort clusters by $|C_j|$;
- 10 Find smallest m s.t. $\sum_{j=1}^m |C_j| \geq \alpha n$;
- 11 $L = \{C_1, \dots, C_m\}$, $S = \{C_{m+1}, \dots, C_c\}$;
- 12 **for** $j \leftarrow 1$ **to** c **do**
- 13 Compute μ_{C_j} ;
- 14 Compute σ_{C_j} according to Eq. (9);
- 15 **end**
- 16 **for** $i \leftarrow 1$ **to** n **do**
- 17 Compute $d(\mathbf{x}_i)$ according to Eq. (10);
- 18 Compute $r(\mathbf{x}_i)$ according to Eq. (11);
- 19 $s(\mathbf{x}_i) \leftarrow d(\mathbf{x}_i) \cdot r(\mathbf{x}_i)$;
- 20 **end**
- 21 **return** s ;

5. Experiments

In this section, a systematic and comprehensive analysis of the MKCOD algorithm is conducted to evaluate its effectiveness and adaptability.

5.1. Experimental preparations

To evaluate the outlier detection performance of the proposed algorithm, 24 OD datasets were collected from the public Web¹. These datasets have been widely used in existing comparative studies. Table 2 summarizes the basic characteristics of the datasets, including the dataset name, the number of samples, the number of attributes, and the number of ground-truth outliers.

As shown in Table 2, the datasets vary substantially in scale, dimensionality, and outlier ratio. The number of samples ranges from 129 to 11183, and the maximum feature dimensionality is 400, while the outlier ratio ranges from 1.2% to 31.6% across different datasets.

To enable a comprehensive comparison, three evaluation metrics are adopted in this study: the Receiver Operating Characteristic (ROC) curve [36], the Area Under the Curve (AUC) [37] and the F1-score [38]. ROC is a widely used evaluation metric in outlier

¹ <https://github.com/BELLoney/Outlier-detection>

Table 2
Experimental datasets.

No.	Datasets	Abbr.	#Objects	#Attributes	Outliers (ratio)
1	Arrhythmia_variant1	Arr	452	279	66 (14.6%)
2	Ecoli	Eco	336	7	9 (2.7%)
3	Pendigits	Pen	6870	16	156 (2.3%)
4	Musk	Musk	3062	166	97 (3.2%)
5	Vertebral	Vert	240	6	30 (12.5%)
6	Wine	Wine	129	13	10 (7.8%)
7	Wpbc_variant1	WPBC	198	33	47 (23.7%)
8	Waveform_0_100_variant1	Wave	3443	21	100 (2.9%)
9	Wdbc_M_39_variant1	WDBC	396	31	39 (9.8%)
10	Mushroom_p_365_variant1	Mush-365	4573	22	365 (8.0%)
11	Autos_variant1	Autos	205	25	25 (12.2%)
12	Mushroom_p_467_variant1	Mush-467	4675	22	467 (10.0%)
13	Breast_cancer_variant1	BC	286	9	85 (29.7%)
14	Cardiotocography_2and3_33_variant1	CTG	1688	21	33 (2.0%)
15	Cardio	Cardio	1831	21	176 (9.6%)
16	Glass	Glass	214	9	9 (4.2%)
17	Optdigits	Opt	5216	64	150 (2.9%)
18	Satellite	Sat	6435	36	2036 (31.6%)
19	Satimage2	Sat-2	5803	36	71 (1.2%)
20	Mnist	MNIST	7603	100	700 (9.2%)
21	Mammography	Mammo	11,183	6	260 (2.3%)
22	Speech	Speech	3686	400	61 (1.7%)
23	Letter	Letter	1600	32	100 (6.3%)
24	Anthyroid	Ann	7200	6	534 (7.4%)

Table 3
Detailed information of compared algorithms.

No.	Names (Year)	Time complexities	Parameters	Ranges of parameters
1	LOF (2000) [39]	$O(n^2 d)$	k	[2, 60]
2	KNN (2000) [40]	$O(n^2 d)$	k	[2, 60]
3	CBLOF (2003) [20]	$O(Inc d)$	c, α, β	[1, 30]; [0.50, 1.00]; 5
4	KPCA (2007) [41]	$O(n^3 + n^2 d)$	κ, γ	RBF; $1/d$
5	HBOS (2012) [42]	$O(nd)$	k	[2, 60]
6	IForest (2012) [43]	$O(T \psi \log \psi)$	T	100
7	AutoEncoder (2014) [44]	$O(EnC_{AE})$	Default	Default
8	DeepSVDD (2018) [45]	$O(EnC_{\phi})$	Default	Default
9	DAGMM (2018) [46]	$O(EnC_{DAGMM})$	Default	Default
10	LMKAD (2019) [8]	$O(Mn^2 d + n^3)$	c, σ	$\{10^{-2}, \dots, 10^2\}$; $\{2^{-6}, \dots, 2^6\}$
11	COPOD (2020) [47]	$O(nd)$	None	None
12	ECOD (2022) [48]	$O(nd)$	None	None
13	DFNO (2024) [49]	$O(n^2)$	k	[2, 60]
14	GBDO (2025) [50]	$O(n^{3/2} + g^2)$	k	[2, 60]
15	GDAD (2025) [51]	$O(dn^2)$	p	{0.1, 0.3, 0.5, 0.7, 0.9}
16	MKCOD (Ours)	$O(Mn^2 d + In^3 + IMcn^2)$	c, α	[1, 30]; [0.50, 1.00]

detection due to its monotonicity and interpretability. Better detection performance corresponds to an ROC curve closer to the upper-left corner. However, when the ROC curves of different algorithms are similar, it is difficult to distinguish their performance using ROC alone. To address this issue, AUC is employed as a quantitative measure of overall performance. The AUC value ranges from 0 to 1, with larger values indicating better performance. In addition, the F1-score jointly considers precision and recall, and is used to further evaluate the detection performance under threshold-based decision settings.

To ensure fairness and comprehensiveness in experimental comparison, the proposed MKCOD algorithm is compared with 15 representative OD algorithms. Specifically, the compared algorithms include Density-Based Local Outlier Factor (LOF) [39], K-Nearest Neighbor-based Outlier Detection (KNN) [40], Cluster-Based Local Outlier Factor (CBLOF) [20], Isolation-Based Outlier Detection (IForest) [43], Histogram-Based Outlier Detection (HBOS) [42], Copula-Based Outlier Detection (COPOD) [47], Empirical Cumulative Distribution Function-based Outlier Detection (ECOD) [48], Local Granular-Ball Density-based Outlier Detection (GBDO) [50], Fuzzy Neighborhood-based Outlier Detection (DFNO) [49], and Fuzzy Granule Density-based Anomaly Detection (GDAD) [51], as well as Kernel Principal Component Analysis for novelty detection (KPCA) [41], Localized Multiple Kernel Learning for Anomaly Detection (LMKAD) [8], Deep Support Vector Data Description (DeepSVDD) [45], Deep Autoencoding Gaussian Mixture Model (DAGMM) [46], and AutoEncoder-based outlier detection (AutoEncoder) [44]. These methods cover a wide range of mainstream outlier detection paradigms, including density-based, distance-based, cluster-based, granular computing-based, fuzzy-based, statistical-based, kernel-based, multi-kernel learning-based, and deep learning-based algorithms.

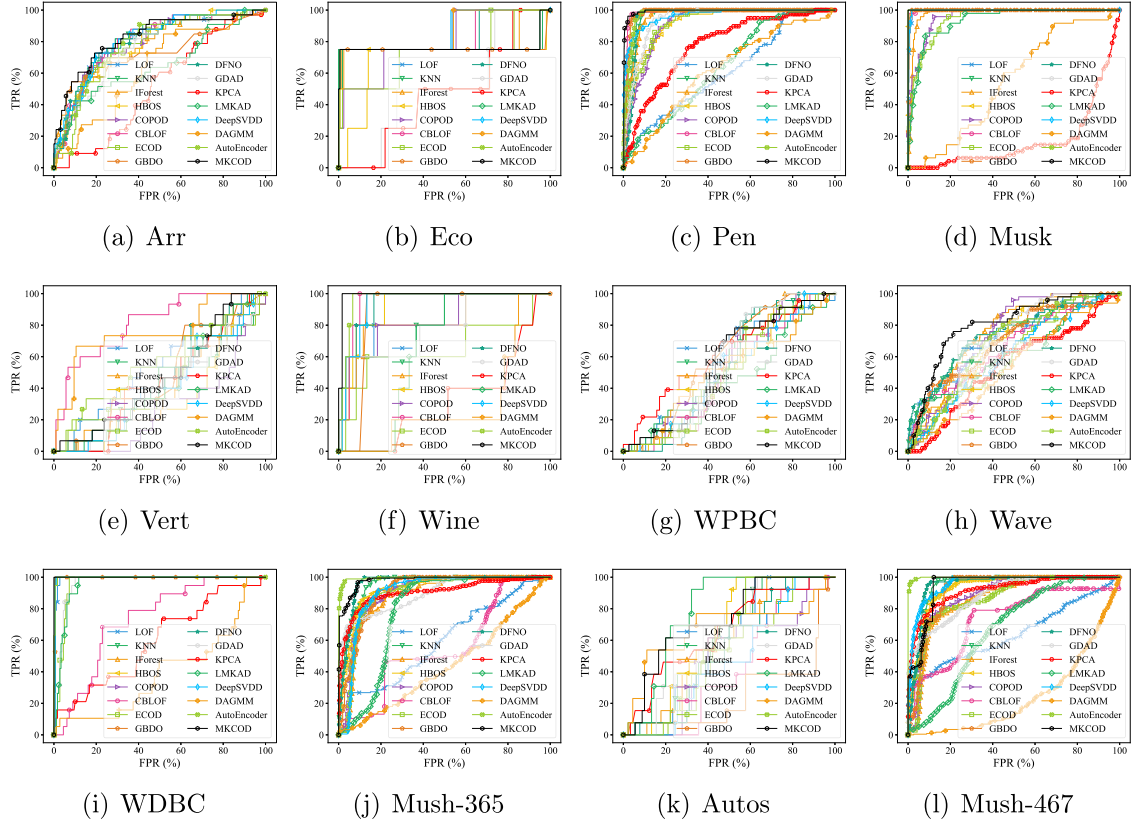


Fig. 3. ROC curves on selected datasets, part 1.

Regarding data partitioning, all samples are randomly divided into equally sized training and test sets using stratified sampling. This random split is repeated three times, and the average test results are reported. For each split, DeepSVDD, DAGMM, AutoEncoder, and IForest are trained only on the normal samples in the training set and then evaluated on the test set. For the remaining shallow methods, parameters are selected from predefined candidate ranges based on their performance on the training set. The selected parameters are then fixed and evaluated on the corresponding test set.

The parameter settings and candidate ranges of all compared algorithms are summarized in Table 3. For the shallow methods requiring parameter search, k and c are varied with a step size of 1, while α is varied with a step size of 0.05. For DeepSVDD, DAGMM, and AutoEncoder, we use the open-source implementations provided by PyOD or ADBench and adopt their corresponding default hyperparameter settings. For multi-kernel construction, seven RBF kernels with $\gamma \in \{0.01, 0.05, 0.1, 1, 10, 50, 100\}$, four polynomial kernels with $(a, d) \in \{(0, 2), (0, 4), (1, 2), (1, 4)\}$, and one linear kernel are employed. All kernel matrices are centered and normalized.

5.2. Analysis on ROC

Based on the parameter settings in Table 3, all algorithms are evaluated on each dataset. For visual comparison, the ROC curves from the first random split are shown in Figs. 3 and 4, where the black curve denotes the proposed MKCOD algorithm.

On Arr, Pen, Opt, and Sat, the ROC curve of MKCOD is closer to the upper-left corner, indicating higher true positive rates at lower false positive rates. On some datasets, the ROC curves of different algorithms are close to each other or overlap. Therefore, visual inspection of ROC curves alone is not sufficient to clearly distinguish the performance of different algorithms.

Accordingly, AUC and F1-score are further reported to provide quantitative evaluation and complementary analysis of detection performance.

5.3. Analysis on AUC

This section further presents a comparative analysis of the AUC values obtained by 16 detection algorithms. The detailed AUC comparison results are reported in Table 4, where the best results are highlighted in bold. Based on 24 public datasets, Table 4 provides an intuitive illustration of the proposed detection algorithm's performance. The related analysis is summarized as follows.

Table 4
Comparison of AUC values of 16 outlier detection algorithms.

Dataset	LOF	KNN	IForest	HBOS	COPOD	CBLOF	ECOD	GBDO	DFNO	GDAD	KPCA	LMKAD	DeepSVDD	DAGMM	AutoEncoder	MKCOD
Arr	0.8275	0.8280	0.8084	0.8163	0.8109	0.8157	0.8133	0.7464	0.8256	0.7964	0.5353	0.6026	0.8172	0.6253	0.7916	0.8302
Eco	0.8678	0.8373	0.8504	0.7541	0.8084	0.8571	0.7458	0.8052	0.8731	0.6596	0.4692	0.8341	0.9164	0.8903	0.8691	0.8713
Pen	0.6221	0.9624	0.9616	0.9186	0.9052	0.9799	0.9239	0.9718	0.9471	0.9338	0.7715	0.7151	0.9352	0.6265	0.9758	0.9943
Musk	0.9933	1.0000	0.9528	1.0000	0.9556	0.9996	0.9519	0.9988	0.9998	1.0000	0.1667	0.9582	0.9959	0.4923	1.0000	1.0000
Vert	0.4834	0.3448	0.3998	0.3281	0.2760	0.6775	0.4546	0.3875	0.4002	0.3865	0.3649	0.4690	0.3306	0.6734	0.5403	0.4013
Wine	0.8911	0.8700	0.8856	0.8800	0.8256	0.6978	0.6844	0.9289	0.9533	0.8489	0.6117	0.6933	0.9200	0.2889	0.9522	0.9878
WPBC	0.5039	0.5596	0.5534	0.5248	0.5640	0.4884	0.5207	0.5330	0.5323	0.5631	0.5546	0.4225	0.5760	0.5191	0.5355	0.5510
Wave	0.7318	0.7499	0.7287	0.6966	0.7468	0.6919	0.6113	0.7172	0.7567	0.7037	0.5475	0.6370	0.6191	0.5325	0.6668	0.8011
WDBC	0.9938	0.9953	0.9961	0.9971	0.9964	0.4713	0.9638	0.9966	0.9960	0.9737	0.6377	0.8722	0.9957	0.3914	0.9972	0.9981
Mush-365	0.6153	0.9538	0.8825	0.9282	0.9003	0.7517	0.8995	0.9243	0.9555	0.8706	0.8984	0.6200	0.9181	0.4417	0.9972	0.9791
Autos	0.6325	0.6561	0.6724	0.6670	0.6197	0.4231	0.5732	0.5148	0.5755	0.7154	0.5493	0.5168	0.5405	0.7157	0.6724	0.7236
Mush-467	0.6477	0.9449	0.8521	0.9344	0.8842	0.7396	0.8640	0.9146	0.9520	0.8446	0.9035	0.6840	0.9702	0.2647	0.9985	0.9548
BC	0.6949	0.7239	0.7310	0.6587	0.6997	0.5852	0.7084	0.7252	0.7119	0.7064	0.5606	0.6098	0.5937	0.5056	0.7016	0.7227
CTG	0.8345	0.7989	0.8186	0.7150	0.8379	0.7834	0.8633	0.7790	0.7893	0.8488	0.5907	0.7264	0.7649	0.7434	0.7840	0.8524
Cardio	0.8658	0.9480	0.9522	0.8851	0.9276	0.7533	0.9399	0.9602	0.9480	0.9196	0.5445	0.7217	0.9622	0.7531	0.9141	0.9719
Glass	0.7776	0.8402	0.7545	0.7335	0.6876	0.5277	0.6653	0.8705	0.8670	0.7480	0.6352	0.8822	0.6811	0.4398	0.7282	0.8772
Opt	0.5932	0.6448	0.8339	0.8818	0.6841	0.6294	0.6090	0.8687	0.4511	0.7573	0.5107	0.5656	0.5188	0.7675	0.8933	0.9997
Sat	0.5750	0.7320	0.8006	0.8108	0.6396	0.7554	0.5887	0.8179	0.7221	0.8217	0.6543	0.5359	0.6485	0.7446	0.7966	0.8358
Sat-2	0.9719	0.9981	0.9895	0.9700	0.9598	0.9851	0.9476	0.9981	0.9982	0.9508	0.7539	0.5196	0.9540	0.9588	0.9977	0.9934
MNIST	0.8174	0.8644	0.8560	0.7414	0.7777	0.8103	0.7493	0.8669	0.8710	0.8139	0.4981	0.8371	0.8493	0.6763	0.9340	0.9039
Manno	0.8701	0.8556	0.8860	0.8657	0.9105	0.8650	0.9110	0.8414	0.8488	0.9089	0.4926	0.6690	0.8927	0.1736	0.9053	0.8877
Speech	0.6201	0.5307	0.5005	0.4834	0.4960	0.5045	0.4770	0.4697	0.6491	0.4815	0.5049	0.5194	0.5237	0.5514	0.4833	0.4976
Letter	0.8689	0.8810	0.6368	0.5931	0.5617	0.6888	0.5798	0.7487	0.9125	0.5820	0.5232	0.8194	0.5008	0.5001	0.7935	0.7845
Ann	0.6983	0.7256	0.9195	0.8159	0.7695	0.6674	0.7869	0.6703	0.7391	0.8559	0.5275	0.7018	0.8378	0.5777	0.8185	0.6790
Average	0.7499	0.8019	0.8010	0.7750	0.7602	0.7145	0.7430	0.7940	0.8031	0.7788	0.5753	0.6722	0.7609	0.5772	0.8228	0.8374

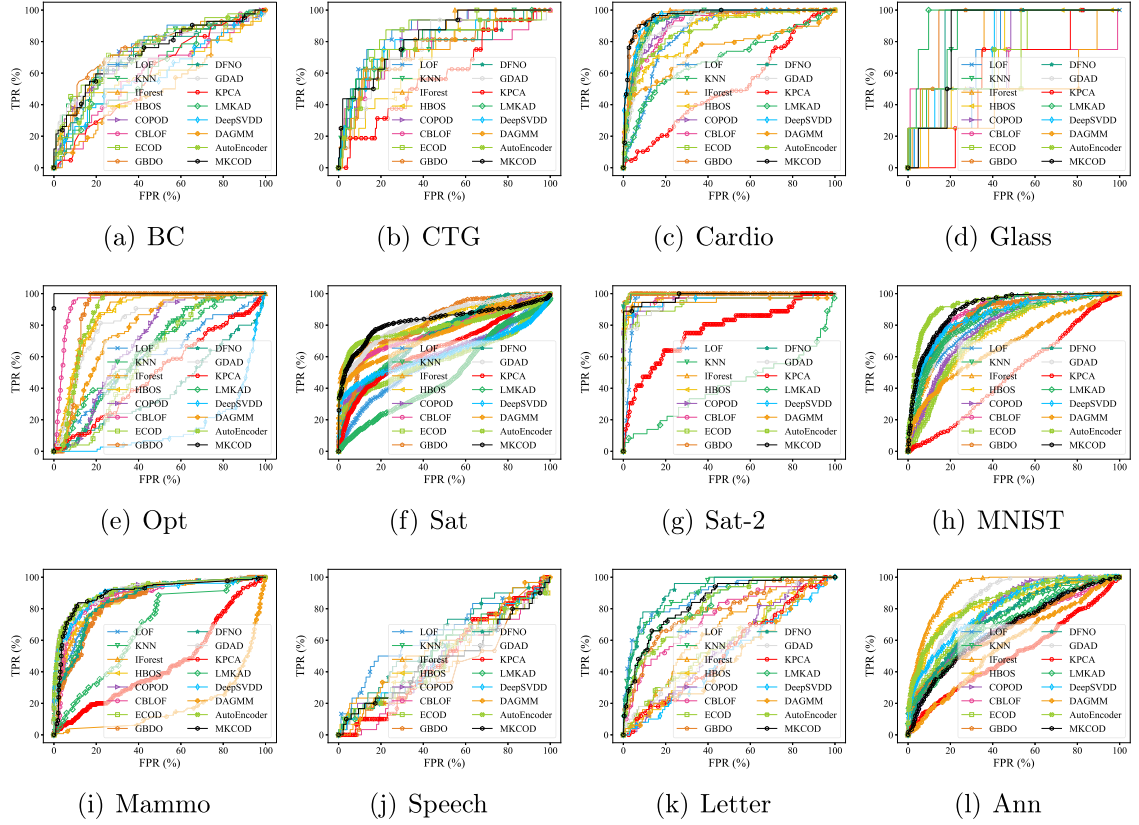


Fig. 4. ROC curves on selected datasets, part 2.

- (1) From the results on individual datasets, MKCOD achieves high AUC values on several datasets. For example, on the Pen dataset, the AUC of MKCOD reaches 0.9943. On Arr, Wave, Wine, Cardio, Opt, and Sat, MKCOD also obtains the best results among the compared methods.
- (2) Some compared methods also perform well on certain datasets. For example, DFNO, IForest, and AutoEncoder obtain higher or comparable AUC values on several datasets. On Speech, several methods obtain AUC values close to or below 0.5, which may be related to its high-dimensional representation and heterogeneous outlier distribution. These results indicate that different methods exhibit varying adaptability to different data distributions.
- (3) From the overall results, according to Table 4, MKCOD achieves an average AUC value of 0.8374, which is the highest among all compared algorithms. This indicates that MKCOD achieves good overall ranking performance across different datasets.

In summary, the AUC results indicate that the proposed MKCOD algorithm shows competitive overall detection performance.

5.4. Analysis on F1-score

This section further presents the F1-score comparison of different detection algorithms. The detailed results are reported in Table 5, where the best results are highlighted in bold. Following the setting in ADBench [52], the decision threshold is determined by the ground-truth outlier ratio of each dataset. Specifically, samples with the highest outlier scores under the corresponding ratio are identified as outliers. The main observations are given below.

- (1) MKCOD obtains relatively high F1-score values on several datasets, such as Pen, Musk, Opt, Wine, WDBC, CTG, and Cardio. This indicates that the proposed method can effectively balance precision and recall on these datasets.
- (2) On some datasets, other methods also achieve higher or comparable F1-score values. This indicates that different methods may show different advantages under different data distributions.
- (3) In terms of the average F1-score, MKCOD achieves an average value of 0.5015, which is the highest among all compared algorithms. This suggests that the proposed method achieves good detection performance under threshold-based evaluation.

In summary, the F1-score results complement the AUC analysis from the perspective of threshold-based evaluation, further indicating that MKCOD achieves competitive detection performance.

Table 5
Comparison of F1-score values of 16 outlier detection algorithms.

Dataset	LOF	KNN	IForest	HBOS	COPOD	CBLOF	ECOD	GBDO	DFNO	GDAD	KPCA	LMKAD	DeepSVD	DAGMM	AutoEncoder	MKCOD
Arr	0.4949	0.4848	0.5051	0.5051	0.4747	0.4545	0.5051	0.5354	0.5152	0.4545	0.1414	0.2424	0.4848	0.2222	0.3434	0.5152
Eco	0.7000	0.5167	0.3833	0.1333	0.0833	0.6000	0.1500	0.4167	0.6833	0.0833	0.0000	0.6167	0.6167	0.5333	0.7667	0.6167
Pen	0.0897	0.1880	0.4188	0.3205	0.2350	0.3974	0.3419	0.3120	0.1368	0.2265	0.0726	0.0727	0.3376	0.0256	0.4701	0.7949
Musk	0.7292	0.9931	0.3681	1.0000	0.4722	0.9653	0.4583	0.9097	0.9931	1.0000	0.0000	0.5417	0.8125	0.0000	1.0000	1.0000
Vert	0.1111	0.0444	0.0222	0.0222	0.0000	0.2889	0.1333	0.0889	0.0444	0.0222	0.0222	0.1111	0.0000	0.3111	0.1111	0.0222
Wine	0.4667	0.4667	0.4000	0.4667	0.6000	0.3333	0.2000	0.2667	0.5333	0.4667	0.2667	0.2000	0.4667	0.0000	0.6000	0.8000
WPBC	0.2132	0.1709	0.1999	0.1854	0.2289	0.1860	0.1570	0.2289	0.1558	0.1425	0.2566	0.2277	0.2283	0.3140	0.1993	0.2138
Wave	0.1800	0.2067	0.1200	0.0467	0.0533	0.1467	0.0533	0.0867	0.2200	0.0533	0.0067	0.0800	0.0533	0.0400	0.0667	0.1267
WDBC	0.8789	0.9316	0.9482	0.9307	0.9316	0.0526	0.6368	0.9658	0.9491	0.7053	0.2053	0.5553	0.9316	0.0518	0.9149	0.9833
Mush-365	0.2441	0.5701	0.3370	0.5100	0.3679	0.3916	0.3424	0.4463	0.5301	0.5082	0.6138	0.1384	0.4536	0.0601	0.9144	0.7760
Autos	0.1282	0.1795	0.1538	0.1795	0.1282	0.0256	0.1282	0.1795	0.0769	0.1538	0.1282	0.0256	0.0256	0.3333	0.1026	0.1795
Mush-467	0.3433	0.4815	0.3775	0.5114	0.4501	0.3932	0.4459	0.4473	0.5798	0.4145	0.6325	0.2208	0.7792	0.0114	0.9444	0.4943
BC	0.5273	0.5434	0.5668	0.4568	0.4808	0.3941	0.4731	0.5670	0.4961	0.5197	0.3381	0.4016	0.3937	0.3228	0.5041	0.5277
CTG	0.0833	0.1642	0.1422	0.1642	0.1017	0.2034	0.2047	0.1029	0.1642	0.0613	0.0404	0.1029	0.1654	0.1017	0.1029	0.2855
Cardio	0.3333	0.5985	0.6136	0.5492	0.5530	0.4091	0.5227	0.7121	0.5871	0.4962	0.1288	0.3636	0.6742	0.4735	0.5985	0.7424
Glass	0.1500	0.1500	0.1500	0.0000	0.1500	0.3333	0.1500	0.2333	0.1500	0.0000	0.0000	0.2333	0.1500	0.0000	0.1500	0.2167
Opt	0.0800	0.0267	0.0578	0.1156	0.0089	0.1422	0.0044	0.0044	0.0222	0.0400	0.0133	0.0622	0.0044	0.0044	0.0044	0.9867
Sat	0.4142	0.5265	0.6441	0.6326	0.4836	0.6185	0.4492	0.6801	0.5079	0.6536	0.4954	0.3481	0.5377	0.5806	0.6997	0.6827
Sat-2	0.1667	0.9167	0.8241	0.6944	0.7130	0.7593	0.6204	0.9074	0.9074	0.5093	0.1389	0.2315	0.8056	0.7315	0.7963	0.8796
MINIST	0.3771	0.4295	0.4019	0.2448	0.2324	0.3562	0.1762	0.4400	0.4410	0.3524	0.0819	0.4067	0.4581	0.2762	0.6143	0.5467
Manno	0.2231	0.2487	0.3154	0.2308	0.4359	0.1051	0.4436	0.2103	0.2256	0.4179	0.0718	0.0590	0.4462	0.0051	0.3769	0.1256
Speech	0.0659	0.0333	0.0222	0.0330	0.0222	0.0219	0.0222	0.0000	0.0878	0.0222	0.0108	0.0222	0.0222	0.0000	0.0222	0.0219
Letter	0.4267	0.3533	0.0867	0.0533	0.0467	0.1600	0.0867	0.3000	0.4200	0.0933	0.0867	0.3533	0.0533	0.0467	0.2600	0.2533
Ann	0.2647	0.2372	0.4432	0.3645	0.2272	0.2385	0.2859	0.2509	0.2522	0.3533	0.1348	0.2859	0.3858	0.0961	0.4245	0.2434
Average	0.3205	0.3943	0.3542	0.3479	0.3117	0.3324	0.2913	0.3872	0.4033	0.3229	0.1620	0.2459	0.3869	0.1892	0.4578	0.5015

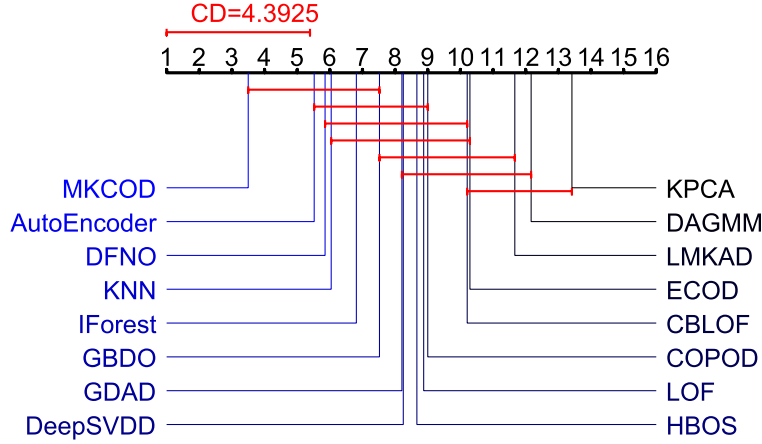


Fig. 5. Nemenyi's test.

5.5. Significance analysis

In this section, following existing studies, we apply the Friedman test and the Nemenyi post-hoc test [53] to assess the statistical significance of performance differences among the 16 comparative algorithms mentioned above. The Friedman test assesses whether there are overall performance differences across multiple algorithms and datasets. The Nemenyi test further identifies which specific algorithm pairs exhibit statistically significant differences.

We conducted a statistical significance analysis on the AUC results of 16 algorithms across 24 datasets, with the significance level set to $\alpha = 0.1$. The Friedman test statistic is $\tau_F = 10.2859$, and the corresponding critical value is 1.5072. Since τ_F exceeds the critical value, the null hypothesis that all algorithms perform equally is rejected. This result indicates statistically significant performance differences among the compared algorithms.

Based on this analysis, the Critical Difference (CD) obtained from the Nemenyi test is 4.3925. As shown in Fig. 5, each point on the horizontal axis represents the average rank of an algorithm, where a smaller rank indicates better AUC performance. If the difference between the average rankings of two algorithms is smaller than the CD, they are connected by a horizontal line, indicating that their performance difference is not statistically significant.

As shown in Fig. 5, no horizontal connections are observed between MKCOD and GDAD, DeepSVDD, LOF, HBOS, COPOD, CBLOF, ECOD, LMKAD, DAGMM or KPCA. This indicates that the performance differences between MKCOD and these algorithms are statistically significant. Except for AutoEncoder, DFNO, KNN, IForest and GBDO, which have relatively close average rankings, MKCOD shows significant performance differences compared with the remaining methods. These results further indicate that MKCOD achieves good overall AUC performance.

Overall, the results of the Friedman test and the Nemenyi post-hoc test indicate that MKCOD achieves good overall performance across multiple datasets, which is generally consistent with the experimental observations.

5.6. Analyses on parameter

To evaluate the stability and adaptability of MKCOD under different parameter settings, the AUC metric is adopted. The interaction between parameters c and α is shown by heatmaps, as shown in Figs. 6 and 7. All heatmaps use a common scale of [0, 1], with lighter yellow colors indicating higher AUC values. The main observations are summarized as follows.

- (1) Different datasets show different sensitivities to parameter combinations. Eco, WDBC, Sat-2, Pen and Musk maintain high AUC values over a relatively wide parameter range. This indicates good parameter stability on these datasets. In contrast, Glass, WPBC, Autos, and Speech show more obvious changes in the heatmaps. This indicates that these datasets are more sensitive to parameter variations.
- (2) From the overall trend, some datasets do not achieve good performance only at a single parameter point. Instead, they form continuous high-AUC regions within certain parameter ranges. This indicates that MKCOD has a certain degree of stability on several datasets.
- (3) Parameters c and α jointly affect the detection performance. The influence of c is related to the data distribution. The favorable number of clusters varies across datasets. Meanwhile, when α is too large, the AUC values on some datasets decrease. This suggests that an unsuitable α value may affect the division between large and small clusters.

In summary, favorable parameter combinations vary across datasets, and no single parameter range is suitable for all datasets. In practical applications, preliminary data can be used to select c and α .

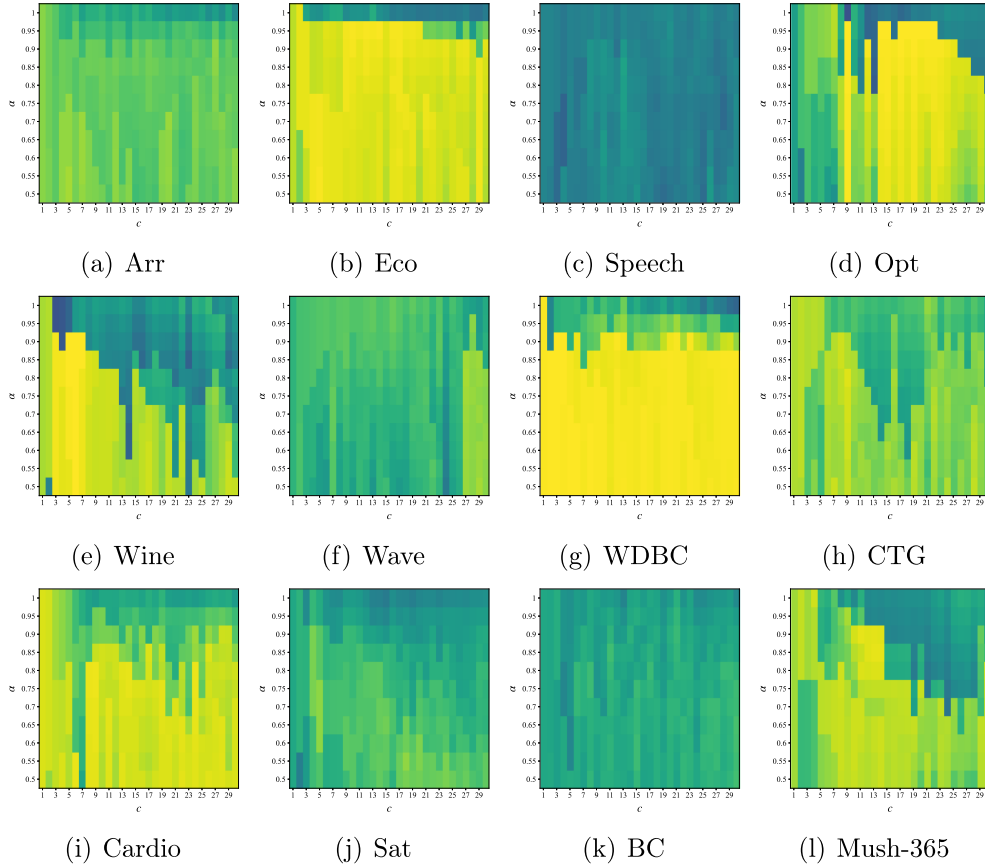


Fig. 6. Parameter analysis on selected datasets, part 1.

5.7. Ablation analysis

To systematically analyze the contribution of each component, four types of ablation experiments are conducted. The first replaces multi-kernel clustering with single-kernel clustering (denoted as SKCOD), where the kernel type is selected from 12 candidate kernels, including one linear kernel, four polynomial kernels with $d \in \{2, 4\}$ and $a \in \{0, 1\}$, and seven RBF kernels with $\gamma \in \{0.01, 0.05, 0.1, 1, 10, 50, 100\}$, while the proposed scoring mechanism is retained. The second retains multi-kernel clustering but replaces the scoring mechanism with the distance to the nearest cluster center (denoted as MK-CenterDist). The third retains multi-kernel clustering but adopts the CBLOF score (denoted as MK-CBLOF). The fourth retains multi-kernel clustering and the proposed scoring mechanism but fixes $\alpha = 1$ (denoted as MKCOD- α).

To ensure a fair comparison, all ablation experiments follow the same experimental protocol as MKCOD. For SKCOD, the kernel type, c , and α are jointly selected on the training set; for MK-CBLOF, c and α are selected; for MK-CenterDist, only c is selected; and for MKCOD- α , α is fixed at 1 and only c is selected. The selected parameters are then evaluated on the corresponding test set. All ablation experiments are conducted on 24 benchmark datasets, and the results are summarized in Table 6, where the best and second-best AUC values are highlighted in bold and underlined, respectively. The following conclusions can be drawn.

- (1) Single-kernel models perform well on some datasets. However, MKCOD achieves higher AUC values than SKCOD on 14 datasets. Its average AUC increases from 0.8182 to 0.8374. These results show that multi-kernel learning improves overall performance and reduces dependence on a single kernel.
- (2) Compared with traditional scoring methods based on cluster-center distance and CBLOF, the proposed outlier scoring mechanism achieves higher AUC values on most datasets. This result supports its effectiveness.
- (3) MKCOD outperforms MKCOD- α , which fixes $\alpha = 1$, on most datasets. In this case, all clusters are treated as large clusters, and the cluster rarity factor is fixed at 1. The score depends only on the normalized distance to the sample's own cluster center. The results show that this special case is less effective than the complete scoring mechanism.

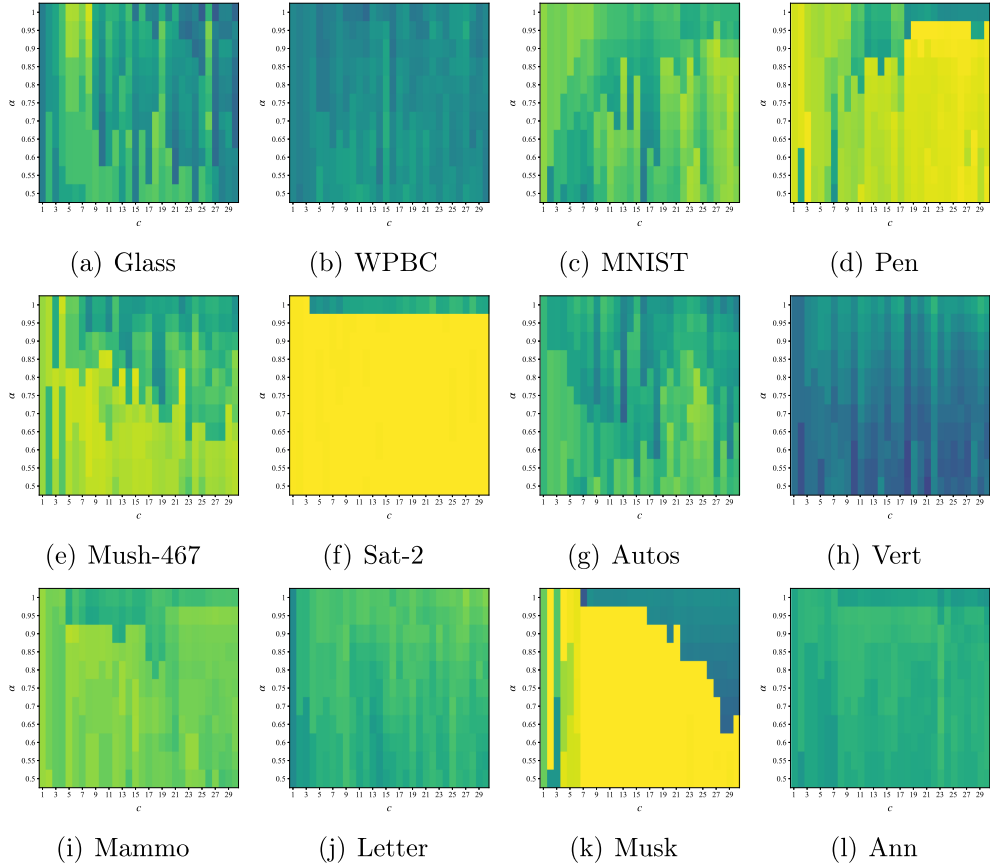


Fig. 7. Parameter analysis on selected datasets, part 2.

Table 6
Ablation experiment results of MKCOD.

Datasets	MK-CenterDist	MK-CBLOF	MKCOD- α	SKCOD	MKCOD
Arr	0.8201	0.8125	0.8201	0.8425	<u>0.8302</u>
Eco	0.8232	0.8968	0.8274	0.8306	<u>0.8713</u>
Pen	<u>0.9826</u>	0.9740	0.9493	0.9737	0.9943
Musk	1.0000	<u>0.9971</u>	1.0000	1.0000	1.0000
Vert	<u>0.4588</u>	0.6639	0.4013	0.4394	0.4013
Wine	0.8189	<u>0.8456</u>	0.7744	0.9878	0.9878
WPBC	0.5229	0.5281	0.5374	0.6192	<u>0.5510</u>
Wave	0.6723	<u>0.7658</u>	0.7373	0.6994	0.8011
WDBC	<u>0.9949</u>	0.4626	<u>0.9949</u>	0.9981	0.9981
Mush-365	0.9398	<u>0.9553</u>	0.9352	0.9249	0.9791
Autos	0.5772	0.6869	0.6108	0.7952	<u>0.7236</u>
Mush-467	0.9166	<u>0.9391</u>	0.8896	0.9134	0.9548
BC	0.6718	0.5753	<u>0.6811</u>	0.6782	0.7227
CTG	0.8476	0.8294	0.8512	0.8729	<u>0.8524</u>
Cardio	0.9437	0.9325	0.9437	<u>0.9516</u>	0.9719
Glass	<u>0.8639</u>	0.7771	0.8538	0.7903	0.8772
Opt	0.8750	<u>0.9093</u>	0.8711	0.8501	0.9997
Sat	0.6845	0.7560	0.6599	<u>0.8290</u>	0.8358
Sat-2	0.9910	0.9806	<u>0.9913</u>	0.9857	0.9934
MNIST	0.8620	<u>0.8995</u>	0.8364	0.8754	0.9039
Mammogram	0.8578	0.8647	0.8578	0.8925	<u>0.8877</u>
Speech	0.4780	0.5776	0.4439	<u>0.5218</u>	0.4976
Letter	0.7983	0.6766	<u>0.7845</u>	0.7306	<u>0.7845</u>
Ann	0.6307	<u>0.6443</u>	0.6293	0.6346	0.6790
Average	0.7930	0.7896	0.7867	<u>0.8182</u>	0.8374

6. Conclusion

This paper proposes a multi-kernel clustering-based outlier detection method and develops the corresponding MKCOD algorithm, which reduces the interference of outliers in the clustering process and enables effective detection of different types of outliers. Extensive comparative experiments with state-of-the-art methods show that the proposed approach achieves competitive performance on multiple datasets in terms of ROC curves, AUC values, and F1-score. The statistical hypothesis tests further provide additional evidence for its overall effectiveness. Parameter analysis reveals the sensitivity of the algorithm to parameter settings, while ablation studies verify that both the multi-kernel clustering strategy and the proposed outlier scoring mechanism are indispensable. Nevertheless, the computational complexity of the proposed method is relatively high, and future work will focus on more efficient optimization and approximation strategies to improve scalability.

CRedit authorship contribution statement

Jiawei Cui: Writing – original draft, Visualization, Software, Methodology, Formal analysis, Conceptualization; **Zhong Yuan:** Writing – review & editing, Supervision, Project administration, Methodology, Funding acquisition, Formal analysis; **Shihao Wang:** Writing – review & editing, Visualization, Validation; **Yinan Chai:** Visualization, Validation, Investigation; **Ze Zhang:** Visualization, Validation, Investigation.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was supported by the [National Natural Science Foundation of China \(62306196\)](#) and Sichuan Science and Technology Program (2026NSFSC0783 and 2024NSFSC0443).

Data availability

Data will be made available on request.

References

- [1] T. Hofmann, B. Schölkopf, A.J. Smola, Kernel methods in machine learning (2008).
- [2] Y. Feng, W. Liang, X. Wan, J. Liu, S. Liu, Q. Qu, R. Guan, H. Xu, X. Liu, Incremental Nyström-based multiple kernel clustering, in: Proceedings of the AAAI Conference on Artificial Intelligence, 39, 2025, pp. 16613–16621.
- [3] C. Ruiz, C.M. Alafiz, J.R. Dorronsoro, A survey on kernel-based multi-task learning, *Neurocomputing* 577 (2024) 127255.
- [4] K.I. Kim, M.O. Franz, B. Scholkopf, Iterative kernel principal component analysis for image modeling, *IEEE Trans. Pattern Anal. Mach. Intell.* 27 (9) (2005) 1351–1366.
- [5] N. Perveen, D. Roy, K.M. Chalavadi, Facial expression recognition in videos using dynamic kernels, *IEEE Trans. Image Process.* 29 (2020) 8316–8325.
- [6] H. Wang, Q. Wang, M. Gao, P. Li, W. Zuo, Multi-scale location-aware kernel representation for object detection, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2018, pp. 1248–1257.
- [7] C. Leslie, E. Eskin, W.S. Noble, The spectrum kernel: a string kernel for SVM protein classification, in: *Biocomputing 2002*, World Scientific, 2001, pp. 564–575.
- [8] C. Gautam, R. Balaji, A. Tiwari, K. Ahuja, et al., Localized multiple kernel learning for anomaly detection: one-class classification, *Knowl.-Based Syst.* 165 (2019) 241–252.
- [9] A. Celikkanat, Y. Shen, F.D. Malliaros, Multiple kernel representation learning on networks, *IEEE Trans. Knowl. Data Eng.* 35 (6) (2022) 6113–6125.
- [10] A.K. Jain, M.N. Murty, P.J. Flynn, Data clustering: a review, *ACM Comput. Surv.* 31 (3) (1999) 264–323.
- [11] C. Zhang, L. Chen, Y.-F. Yu, Y.-P. Zhao, Z. Shi, Y. Wang, W. Bai, Selective multiple kernel fuzzy clustering with locality preserved ensemble, *Knowl.-Based Syst.* 301 (2024) 112327.
- [12] R. Xu, D. Wunsch, Survey of clustering algorithms, *IEEE Trans. Neural Netw.* 16 (3) (2005) 645–678.
- [13] P. Berkhin, A survey of clustering data mining techniques, in: *Grouping Multidimensional Data: Recent Advances in Clustering*, Springer, 2006, pp. 25–71.
- [14] S.A.N. Nozad, M.A. Haeri, G. Folino, SDCOR: Scalable density-based clustering for local outlier detection in massive-scale datasets, *Knowl.-based Syst.* 228 (2021) 107256.
- [15] X. Gao, J. Yu, S. Zha, S. Fu, B. Xue, P. Ye, Z. Huang, G. Zhang, An ensemble-based outlier detection method for clustered and local outliers with differential potential spread loss, *Knowl.-Based Syst.* 258 (2022) 110003.
- [16] V. Chandola, A. Banerjee, V. Kumar, Anomaly detection: a survey, *ACM Comput. Surv.* 41 (3) (2009) 1–58.
- [17] M. Ester, H.-P. Kriegel, J. Sander, X. Xu, et al., A density-based algorithm for discovering clusters in large spatial databases with noise, in: *kdd*, 96, 1996, pp. 226–231.
- [18] R.J. Campello, D. Moulavi, J. Sander, Density-based clustering based on hierarchical density estimates, in: *Pacific-asia Conference on Knowledge Discovery and Data Mining*, Springer, 2013, pp. 160–172.
- [19] W.I.D. Mining, Data mining: concepts and techniques, *Morgan Kaufmann* 10 (559–569) (2006) 4.
- [20] Z. He, X. Xu, S. Deng, Discovering cluster-based local outliers, *Pattern Recognit. Lett.* 24 (9–10) (2003) 1641–1650.
- [21] B. Schölkopf, A. Smola, K.-R. Müller, Nonlinear component analysis as a kernel eigenvalue problem, *Neural Comput.* 10 (5) (1998) 1299–1319.
- [22] H. Zha, X. He, C. Ding, M. Gu, H. Simon, Spectral relaxation for k-means clustering, *Adv. Neural Inf. Process. Syst.* 14 (2001).
- [23] L. Du, P. Zhou, L. Shi, H. Wang, M. Fan, W. Wang, Y.-D. Shen, Robust multiple kernel k-means using l21-norm, in: *IJCAI*, 15, 2015, pp. 3476–3482.
- [24] M. Gönen, E. Alpaydın, Multiple kernel learning algorithms, *J. Mach. Learn. Res.* 12 (2011) 2211–2268.
- [25] S. Yu, L. Tranchevent, X. Liu, W. Glanzel, J.A.K. Suykens, B. De Moor, Y. Moreau, Optimized data fusion for kernel k-means clustering, *IEEE Trans. Pattern Anal. Mach. Intell.* 34 (5) (2011) 1031–1039.

- [26] H.-C. Huang, Y.-Y. Chuang, C.-S. Chen, Multiple kernel fuzzy clustering, *IEEE Trans. Fuzzy Syst.* 20 (1) (2011) 120–134.
- [27] S. Bang, Y. Yu, W. Wu, Robust multiple kernel k-means clustering using min-max optimization, (2018). [arXiv preprint arXiv:1803.02458](https://arxiv.org/abs/1803.02458)
- [28] M. Ankerst, M.M. Breunig, H.-P. Kriegel, J. Sander, OPTICS: Ordering points to identify the clustering structure, *ACM Sigmod Rec.* 28 (2) (1999) 49–60.
- [29] S. Guha, R. Rastogi, K. Shim, ROCK: A robust clustering algorithm for categorical attributes, *Inf. Syst.* 25 (5) (2000) 345–366.
- [30] J.B. McQueen, Some methods of classification and analysis of multivariate observations, in: *Proc. of 5th Berkeley Symposium on Math. Stat. and Prob.*, 1967, pp. 281–297.
- [31] S. Chawla, A. Gionis, K-means-: a unified approach to clustering and outlier detection, in: *Proceedings of the 2013 SIAM International Conference on Data Mining*, SIAM, 2013, pp. 189–197.
- [32] K. Fan, On a theorem of weyl concerning eigenvalues of linear transformations i, *Proc. Natl. Acad. Sci.* 35 (11) (1949) 652–655.
- [33] A. Ng, M. Jordan, Y. Weiss, On spectral clustering: analysis and an algorithm, *Adv. Neural Inf. Process. Syst.* 14 (2001).
- [34] M. Kloft, U. Brefeld, S. Sonnenburg, A. Zien, Lp-norm multiple kernel learning, *J. Mach. Learn. Res.* 12 (2011) 953–997.
- [35] Y. Zhao, Z. Nasrullah, Z. Li, PyOD: a python toolbox for scalable outlier detection, *J. Mach. Learn. Res.* 20 (96) (2019) 1–7.
- [36] Z. Yuan, B. Chen, J. Liu, H. Chen, D. Peng, P. Li, Anomaly detection based on weighted fuzzy-rough density, *Appl. Soft Comput.* 134 (2023) 109995.
- [37] Z. Yuan, H. Chen, C. Luo, D. Peng, MFGAD: Multi-fuzzy granules anomaly detection, *Inf. Fusion* 95 (2023) 17–25.
- [38] C. Van Rijsbergen, *Information retrieval: theory and practice*, in: *Proceedings of the Joint IBM/University of Newcastle upon Tyne Seminar on Data Base Systems*, 79, Butterworth-Heinemann Oxford, UK, 1979, pp. 1–14.
- [39] M.M. Breunig, H.-P. Kriegel, R.T. Ng, J. Sander, LOF: Identifying density-based local outliers, in: *Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data*, 2000, pp. 93–104.
- [40] S. Ramaswamy, R. Rastogi, K. Shim, Efficient algorithms for mining outliers from large data sets, in: *Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data*, 2000, pp. 427–438.
- [41] H. Hoffmann, Kernel PCA for novelty detection, *Pattern Recognit.* 40 (3) (2007) 863–874.
- [42] M. Goldstein, A. Dengel, Histogram-based outlier score (hbos): a fast unsupervised anomaly detection algorithm, *KI-2012: Poster Demo Track 1* (2012) 59–63.
- [43] F.T. Liu, K.M. Ting, Z.-H. Zhou, Isolation-based anomaly detection, *ACM Trans. Knowl. Discov. Data* 6 (1) (2012) 1–39.
- [44] M. Sakurada, T. Yairi, Anomaly detection using autoencoders with nonlinear dimensionality reduction, in: *Proceedings of the MLSDA 2014 2nd Workshop on Machine Learning for Sensory Data Analysis*, 2014, pp. 4–11.
- [45] L. Ruff, R. Vandermeulen, N. Goernitz, L. Deecke, S.A. Siddiqui, A. Binder, E. Müller, M. Kloft, Deep one-class classification, in: *International Conference on Machine Learning*, PMLR, 2018, pp. 4393–4402.
- [46] B. Zong, Q. Song, M.R. Min, W. Cheng, C. Lumezanu, D. Cho, H. Chen, Deep autoencoding gaussian mixture model for unsupervised anomaly detection, in: *International Conference on Learning Representations*, 2018.
- [47] Z. Li, Y. Zhao, N. Botta, C. Ionescu, X. Hu, Copod: copula-based outlier detection, in: *2020 IEEE International Conference on Data Mining (ICDM)*, IEEE, 2020, pp. 1118–1123.
- [48] Z. Li, Y. Zhao, X. Hu, N. Botta, C. Ionescu, G.H. Chen, Ecod: unsupervised outlier detection using empirical cumulative distribution functions, *IEEE Trans. Knowl. Data Eng.* 35 (12) (2022) 12181–12193.
- [49] Z. Yuan, P. Hu, H. Chen, Y. Chen, Q. Li, DFNO: Detecting fuzzy neighborhood outliers, *IEEE Trans. Knowl. Data Eng.* (2024).
- [50] X. Su, X. Wang, D. Peng, X. Song, H. Zheng, Z. Yuan, Identifying outliers via local granular-ball density, *IEEE Trans. Neural Netw. Learn. Syst.* (2025).
- [51] B. Chen, Z. Yuan, D. Peng, X. Chen, H. Chen, Y. Chen, Integrating granular computing with density estimation for anomaly detection in high-dimensional heterogeneous data, *Inf. Sci.* 690 (2025) 121566.
- [52] S. Han, X. Hu, H. Huang, M. Jiang, Y. Zhao, Adbench: anomaly detection benchmark, *Adv. Neural Inf. Process. Syst.* 35 (2022) 32142–32159.
- [53] J. Demšar, Statistical comparisons of classifiers over multiple data sets, *J. Mach. Learn. Res.* 7 (Jan) (2006) 1–30.